

Domain Adaptation su MRI ADNI con VAE Scanner Signatures e CycleGAN

Relazione tecnica del progetto

Gruppo: EventHorizon Track: 8

2026

Sommario

Questa relazione descrive il lavoro svolto per costruire una pipeline di domain adaptation su immagini di risonanza magnetica strutturale provenienti da ADNI. Il problema affrontato e' la variazione di dominio causata da scanner e produttori diversi: un classificatore addestrato su scansioni acquisite da alcuni produttori puo' degradare quando viene valutato su immagini acquisite da un produttore non visto. La pipeline prepara volumi T1, costruisce split patient-level usando due produttori come source domain e un terzo produttore come target domain, addestra un classificatore baseline source-only, integra traduzioni CycleGAN, genera augmentation tramite VAE scanner signatures e usa anche una variante AdaIN per il trasferimento di statistiche di stile scanner. Il codice e' stato organizzato secondo la struttura del repository: preprocessing in `src/datasets`, architetture in `src/models`, training in `src/training`, valutazione in `src/evaluation`, checkpoint in `experiments/checkpoints`, log in `experiments/logs` e figure in `figures`. Nella valutazione finale multi-modello, addestrando su SIEMENS e GE MEDICAL SYSTEMS e testando su PHILIPS MEDICAL SYSTEMS, il baseline `source_only` raggiunge accuracy 0.6571 e macro-F1 0.3966; `cyclegan_augmented` raggiunge 0.4286 e 0.4281; `signature_gan_augmented` raggiunge 0.6286 e 0.3860; `adain_gan_augmented` raggiunge 0.7143 e 0.6500. In questa configurazione AdaIN+GAN e' il modello migliore sul target PHILIPS.

Indice

1	Introduzione	4
2	Obiettivo del progetto	4
3	Organizzazione del repository	5
4	Dataset e dati utilizzati	6
5	Preprocessing	6

6	Split source e target	7
7	Architetture implementate	8
7.1	Classificatore CNN	8
7.2	VAE per firme latenti di scanner	8
7.3	AdaIN per firme di stile scanner	9
7.4	CycleGAN	10
8	Problemi tecnici risolti durante lo sviluppo	11
8.1	Dipendenza NIfTI	11
8.2	Dimensioni diverse delle slice	11
8.3	Caricamento lento dei volumi	11
8.4	Separazione delle responsabilita'	11
9	Pipeline sperimentale	11
9.1	Estrazione dati	11
9.2	Preparazione volumi	12
9.3	Split domini	12
9.4	Training VAE scanner signatures	12
9.5	Generazione immagini con firme sintetiche	12
9.6	Training AdaIN scanner signatures	13
9.7	Generazione immagini con AdaIN	13
9.8	Training CycleGAN	13
9.9	Traduzione source-to-target	13
9.10	Training set aumentato VAE+GAN	14
9.11	Training set aumentato AdaIN+GAN	14
9.12	Training classificatore finale	14
9.13	Ambiente di esecuzione	15
10	Valutazione	15
11	Risultati quantitativi	16
12	Scatterplot delle metriche	17
13	Curve temporali del classificatore	18

14 Curve temporali della CycleGAN	18
15 Preview qualitative delle immagini modificate	19
16 Figure qualitative aggiuntive	20
17 Discussione dei risultati	24
18 Checkpoint e artefatti prodotti	25
19 Riproducibilita'	27
20 Limiti	28
20.1 Dimensione del dataset	28
20.2 Uso di slice 2D	28
20.3 Modelli generativi non supervisionati	28
20.4 Valutazione qualitativa non clinica	28
20.5 Assenza di confronti multipli	28
21 Possibili estensioni	29
22 Conclusione	29
A Comandi principali e spiegazione operativa	30
A.1 Accesso al cluster e ambiente GPU	30
A.2 Esecuzione automatica completa	30
A.3 Preparazione ADNI	30
A.4 Split patient-level per dominio scanner	31
A.5 Baseline source-only	31
A.6 CycleGAN e traduzione source-to-target	31
A.7 Training del classificatore CycleGAN-augmented	32
A.8 VAE scanner signatures	32
A.9 Training set VAE+GAN e classificatore finale	33
A.10 Ramo AdaIN predisposto	33
A.11 Valutazione e figure in docs/figures/	34

1 Introduzione

La robustezza dei modelli di deep learning in ambito medico dipende in modo critico dalla qualita' e dall'omogeneita' dei dati di training. Nel caso delle immagini di risonanza magnetica, anche quando il protocollo clinico e' simile, le immagini possono cambiare sensibilmente in funzione del produttore dello scanner, della forza di campo, dei parametri di acquisizione e della pipeline di preprocessing. Questa variazione prende il nome di *domain shift*: il modello apprende correlazioni statistiche sul dominio di addestramento, ma tali correlazioni possono non trasferirsi direttamente a un dominio di test differente.

Il progetto realizzato studia questo problema usando scansioni strutturali MRI del dataset ADNI. L'obiettivo pratico e' costruire una pipeline riproducibile che:

1. legge gli archivi MRI e i metadati IDA;
2. filtra scansioni T1 e classi diagnostiche di interesse;
3. costruisce split a livello paziente;
4. considera il produttore dello scanner come dominio;
5. usa due produttori per il training e un terzo produttore per il test;
6. addestra un VAE per estrarre firme latenti dei macchinari;
7. aggiunge AdaIN per trasferire statistiche di stile scanner nelle feature;
8. genera immagini sintetiche con entrambe le strategie;
9. integra anche traduzioni CycleGAN source-to-target;
10. addestra un classificatore sul dataset aumentato;
11. confronta quantitativamente i risultati sul produttore non visto.

La scelta di confrontare VAE, AdaIN e CycleGAN e' motivata da esigenze complementari. Il VAE consente di lavorare in uno spazio latente compatto, dove il produttore dello scanner puo' essere rappresentato come una firma media. AdaIN interviene invece sulle statistiche di feature, separando piu' esplicitamente contenuto anatomico e stile scanner. La CycleGAN consente infine di produrre una traduzione image-to-image non appaiata verso il dominio target, usando immagini target senza supervisioni di classe. Questo e' coerente con molti scenari reali di domain adaptation in cui si possiedono immagini non etichettate dal dominio target, ma non necessariamente annotazioni diagnostiche complete.

2 Obiettivo del progetto

L'obiettivo principale e' valutare se un training set aumentato con firme sintetiche di scanner, generate tramite VAE o AdaIN, e con traduzioni CycleGAN possa migliorare la classificazione CN vs AD su un produttore non visto. Il progetto non mira a proporre

un modello clinico pronto all'uso, ma una pipeline sperimentale completa, verificabile e coerente con la struttura richiesta dal repository.

La domanda sperimentale puo' essere formulata come segue:

Dato un classificatore addestrato su MRI provenienti da scanner SIEMENS e GE MEDICAL SYSTEMS, quale strategia di augmentation fra VAE scanner signatures, AdaIN scanner signatures e traduzione CycleGAN generalizza meglio su un target test set PHILIPS MEDICAL SYSTEMS?

La risposta empirica, nella run finale disponibile, e' che la pipeline e' eseguibile end-to-end e che il ramo AdaIN+GAN e' l'unico a migliorare chiaramente il controllo source-only. Il baseline raggiunge accuracy 0.6571 e macro-F1 0.3966 su PHILIPS; CycleGAN-augmented ottiene 0.4286/0.4281; VAE+GAN-augmented ottiene 0.6286/0.3860; AdaIN+GAN-augmented sale a 0.7143/0.6500. Questo risultato mostra una pipeline funzionante e mette in evidenza limiti tecnici e sperimentali importanti: numero ridotto di campioni, training generativo non supervisionato, uso di una sola slice centrale per volume, assenza di validazione qualitativa clinica e architetture volutamente leggere.

3 Organizzazione del repository

Durante il lavoro e' stata rispettata l'architettura del repository `dl26-projects`. Ogni cartella contiene componenti coerenti con il proprio README:

- `data/`: archivi ADNI, dati grezzi estratti e dati processati locali;
- `src/datasets/`: script di preparazione del dataset, split e costruzione del training set aumentato;
- `src/models/`: definizioni delle architetture PyTorch;
- `src/training/`: script di training e traduzione;
- `src/evaluation/`: metriche, confronto degli esperimenti e generazione figure;
- `experiments/configs/`: configurazioni sperimentali;
- `experiments/checkpoints/`: checkpoint addestrati;
- `experiments/logs/`: log CSV delle curve temporali;
- `experiments/outputs/`: metriche e output degli esperimenti;
- `docs/figures/`: figure usate nella relazione;
- `docs/`: report e documentazione finale.

Questa separazione evita di mischiare responsabilita' diverse. Le classi dei modelli non sono lasciate negli script di training, ma collocate in `src/models`. Le metriche e le figure non vengono prodotte manualmente, ma da uno script in `src/evaluation`. I checkpoint non sono salvati in `src/models`, perche' quella cartella contiene codice sorgente e non pesi addestrati.

4 Dataset e dati utilizzati

Il dataset utilizzato e' una selezione ADNI composta da scansioni MRI in formato NIfTI e metadati IDA in formato XML. Gli input locali usati dalla pipeline sono:

- `data/ADNI1_Annual_2_Yr_3T.zip`;
- `data/ADNI1_Annual_2_Yr_3T_IDA_Metadata.zip`;
- `data/ADNI1_Annual_2_Yr_3T_5_26_2026.csv`.

Il CSV di collezione contiene le colonne necessarie per associare ogni immagine al soggetto, al gruppo diagnostico, alla descrizione della scansione e alla data di acquisizione:

- Image Data ID;
- Subject;
- Group;
- Description;
- Acq Date.

Sono stati estratti 306 file `.nii` e 612 file `.xml`. Dopo il filtraggio per scansioni T1, field strength 3T e classi diagnostiche CN/AD, il dataset preparato contiene 173 volumi relativi a 52 soggetti.

Tabella 1: Distribuzione dei dati preparati per produttore e classe.

Produttore	Disease	Healthy
GE Medical Systems	9	18
Philips Medical Systems	23	48
Siemens	26	49
Totale	58	115

La tabella mostra che le classi sono sbilanciate verso *healthy*; inoltre i produttori hanno numerosita' diverse. Questo influenza sia la stabilita' del training sia l'interpretazione delle metriche. Per questo motivo, oltre all'accuracy, viene riportata la macro-F1, che pesa le classi in modo piu' bilanciato.

5 Preprocessing

Il preprocessing e' implementato in `src/datasets/prepare_adni.py`. Lo script svolge diverse operazioni:

1. legge il CSV di collezione ADNI;
2. normalizza gli identificativi immagine;
3. inferisce la modalita' dalla descrizione;
4. filtra le scansioni T1;
5. assegna l'etichetta diagnostica in modalita' `cn_vs_ad`;
6. collega il record CSV al file NIfTI e al file XML;
7. estrae produttore, modello scanner, field strength e series UID dai metadati XML;
8. normalizza il volume tramite min-max normalization;
9. salva il volume come `.npy`;
10. produce `index_prepared.csv`.

Durante l'esecuzione iniziale e' emerso un collo di bottiglia: cercare ricorsivamente file NIfTI e XML per ogni riga del CSV era troppo lento. Lo script e' stato quindi ottimizzato indicizzando una sola volta tutti i file disponibili. Questa modifica riduce drasticamente il costo della fase di matching.

La normalizzazione min-max e' definita come:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Quando il volume ha valore massimo uguale al minimo, viene restituito un volume nullo per evitare divisioni per zero.

6 Split source e target

Gli split sono creati con `src/datasets/make_domain_splits.py`. Il produttore dello scanner viene usato come dominio. Nella pipeline finale lo script e' stato esteso per accettare piu' produttori source, cosi' da addestrare su due macchinari e testare su un terzo produttore lasciato fuori dal training:

- source domains: SIEMENS e GE MEDICAL SYSTEMS;
- target domain: PHILIPS MEDICAL SYSTEMS.

Lo split e' patient-level: un soggetto non deve comparire contemporaneamente in training e test. Questa scelta e' essenziale per evitare leakage, perche' piu' scansioni dello stesso soggetto possono essere molto simili e rendere il task artificialmente piu' semplice.

Tabella 2: Split patient-level usati nella pipeline.

Split	Scansioni	Soggetti	Dominio
Source train	81	24	Siemens + GE
Source validation	21	7	Siemens + GE
Target adaptation unlabeled	36	10	Philips
Target test	35	11	Philips

La parte target adaptation viene usata per addestrare o applicare la componente CycleGAN come dominio target non etichettato. Il target test set viene mantenuto separato e usato solo per la valutazione finale dei classificatori.

7 Architetture implementate

Le architetture sono state collocate in `src/models`, in accordo con il README della cartella.

7.1 Classificatore CNN

Il classificatore e' definito in `src/models/classifier.py`. Si tratta di una CNN leggera che opera su slice 2D monocanale estratte dal centro del volume MRI. La rete e' composta da:

- tre blocchi convolutional con batch normalization e ReLU;
- max pooling nei primi due blocchi;
- adaptive average pooling a dimensione fissa;
- fully connected layer da 128 unita';
- dropout;
- layer di output a due classi.

L'uso di una rete leggera e' stato scelto per rendere il training praticabile localmente e per mantenere la pipeline semplice. Il limite principale e' che una singola slice centrale non rappresenta tutta l'informazione volumetrica del cervello. Un miglioramento naturale sarebbe passare a modelli 2.5D o 3D.

7.2 VAE per firme latenti di scanner

La componente VAE e' definita in `src/models/vae.py` e addestrata con `src/training/train_vae_signature.py`. L'obiettivo non e' classificare direttamente la malattia, ma

proiettare le slice MRI in uno spazio latente in cui sia possibile stimare una firma media per ciascun produttore source.

Il modello usa un encoder convoluzionale che produce media e log-varianza:

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x)^2)$$

Il decoder ricostruisce la slice a partire dal vettore latente campionato. La loss e':

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \beta \mathcal{L}_{KL}$$

dove $\mathcal{L}_{rec}$ e' la mean squared error fra ricostruzione e input, mentre $\mathcal{L}_{KL}$ regolarizza la distribuzione latente verso una normale standard. Nella run finale sono state usate 40 epoche, batch size 8, latent dimension 64 e $\beta = 10^{-3}$.

Dopo il training, per ogni produttore m viene calcolata una firma media:

$$s_m = \frac{1}{N_m} \sum_{i=1}^{N_m} \mu_\phi(x_i)$$

Per generare una versione sintetica di una immagine source x , si codifica l'immagine in z , si somma la direzione fra la firma target e quella source e si ottiene un vettore spostato:

$$z' = z + \alpha(s_{target} - s_{source}) + \epsilon$$

Nella prima implementazione veniva salvato direttamente $Decoder(z')$. Questa scelta si e' rivelata fragile: il decoder VAE, addestrato su pochi campioni, tendeva a generare immagini molto scure e poco realistiche. La generazione e' stata quindi corretta in forma residuale:

$$\hat{x} = clip(x + \lambda(Decoder(z') - Decoder(z)), 0, 1)$$

In questo modo $Decoder(z') - Decoder(z)$ viene interpretato come variazione scanner-style stimata dal VAE, mentre il contenuto anatomico principale resta quello della slice reale x . Nella run aggiornata sono stati usati $\alpha = 1.0$, $\lambda = 0.6$ e `noise-scale=0.10`. Questa scelta e' piu' conservativa: evita il collasso visivo del decoder e produce augmentation piu' adatte al training del classificatore.

7.3 AdaIN per firme di stile scanner

La variante AdaIN e' definita in `src/models/adain.py`. Gli script di training e generazione sono `train_adain_signature.py` e `generate_adain_augmented.py`. L'idea e' evitare di chiedere al modello di generare tutta l'immagine partendo da un vettore latente globale, come nel VAE, e trasferire invece lo stile scanner tramite statistiche di feature.

Dato un tensore di feature f , AdaIN normalizza il contenuto e lo ri-scala con media e deviazione standard dello stile target:

$$AdaIN(f, \mu_s, \sigma_s) = \sigma_s \frac{f - \mu(f)}{\sigma(f)} + \mu_s$$

Nel nostro caso μ_s e σ_s sono calcolate come firme medie per produttore sulle feature dell'encoder. Il decoder ricostruisce poi la slice a partire dalle feature con statistiche target. Rispetto al VAE, questa formulazione e' piu' coerente con l'ipotesi che anatomia e scanner style siano fattori separabili: il contenuto resta nelle feature normalizzate, mentre il produttore agisce su media e varianza canale per canale.

La pipeline AdaIN produce un CSV dedicato, `data/processed/05_adain_signature_augmented/source_train_adain_signature_augmented.csv`, che puo' essere fuso con le traduzioni GAN nello stesso modo del dataset VAE. Il classificatore risultante viene salvato in `experiments/outputs/classifier_adain_gan_augmented`, cosi' da confrontarlo direttamente con `classifier_signature_gan_augmented`.

7.4 CycleGAN

La CycleGAN e' definita in `src/models/cyclegan.py`. Sono presenti due componenti:

- **Generator:** traduce immagini da un dominio all'altro;
- **Discriminator:** distingue immagini reali da immagini generate.

Durante il training vengono usati due generatori:

$$G_{S \rightarrow T}$$

$$G_{T \rightarrow S}$$

e due discriminatori, uno per il source domain e uno per il target domain. La loss totale del generatore include:

- adversarial loss;
- cycle-consistency loss;
- identity loss.

La cycle-consistency impone che una immagine source tradotta nel target e poi riportata nel source rimanga coerente:

$$G_{T \rightarrow S}(G_{S \rightarrow T}(x_S)) \approx x_S$$

Analogamente per il target:

$$G_{S \rightarrow T}(G_{T \rightarrow S}(x_T)) \approx x_T$$

8 Problemi tecnici risolti durante lo sviluppo

Durante l'esecuzione della pipeline sono emersi diversi problemi pratici.

8.1 Dipendenza NIfTI

Il sistema inizialmente non aveva `nibabel`, libreria necessaria per caricare i file NIfTI. La dipendenza e' stata installata e verificata. Dopo l'installazione, `prepare_adni.py` e' stato in grado di leggere i volumi e salvarli in formato NumPy.

8.2 Dimensioni diverse delle slice

Il training del classificatore si e' inizialmente fermato perche' alcune slice avevano dimensioni diverse, ad esempio 240×256 e 256×256 . PyTorch non puo' creare batch con tensori di dimensioni diverse. Il problema e' stato risolto aggiungendo un resize bilineare a 256×256 nei dataset usati da classificatore, CycleGAN e traduzione.

8.3 Caricamento lento dei volumi

Un altro collo di bottiglia era il caricamento dell'intero volume 3D solo per estrarre una slice centrale. Lo script e' stato ottimizzato usando `np.load(..., mmap_mode="r")`, cosi' da ridurre il carico di I/O.

8.4 Separazione delle responsabilita'

Inizialmente alcune definizioni modello erano dentro gli script di training. Per allineare il progetto al README di `src/models`, le architetture sono state spostate in file dedicati e importate dagli script di training.

9 Pipeline sperimentale

La pipeline finale e' composta da una fase comune di preparazione/split e da due rami confrontabili: VAE+GAN e AdaIN+GAN. Le fasi iniziali di estrazione e preparazione dei volumi rimangono invariate rispetto alla pipeline CycleGAN originale; la parte nuova riguarda split con due produttori source, generazione di firme sintetiche con VAE o AdaIN e fusione con le traduzioni GAN.

9.1 Estrazione dati

Gli archivi sono stati estratti in:

```
data/raw/nifti
data/raw/metadata
```

Il conteggio finale e' stato:

- 306 file NIfTI;
- 612 file XML.

9.2 Preparazione volumi

```
python3 src/datasets/prepare_adni.py \  
  --csv data/ADNI1_Annual_2_Yr_3T_5_26_2026.csv \  
  --nifti-root data/raw/nifti \  
  --metadata-root data/raw/metadata \  
  --out-dir data/processed/00_prepared \  
  --field 3T \  
  --label-mode cn_vs_ad
```

Il risultato principale e':

```
data/processed/00_prepared/index_prepared.csv
```

9.3 Split domini

```
python3 src/datasets/make_domain_splits.py \  
  --index-csv data/processed/00_prepared/index_prepared.csv \  
  --source-manufacturers "SIEMENS,GE_MEDICAL_SYSTEMS" \  
  --target-manufacturer "PHILIPS_MEDICAL_SYSTEMS" \  
  --out-dir data/processed/01_domain_split
```

9.4 Training VAE scanner signatures

```
python3 src/training/train_vae_signature.py \  
  --train-csv data/processed/01_domain_split/source_train.csv \  
  --out-dir experiments/outputs/vae_signature \  
  --checkpoint-dir experiments/checkpoints \  
  --checkpoint-name vae_signature.pt \  
  --epochs 40 \  
  --batch-size 8
```

9.5 Generazione immagini con firme sintetiche

```
python3 src/training/generate_signature_augmented.py \  
  --source-csv data/processed/01_domain_split/source_train.csv \  
  --vae experiments/checkpoints/vae_signature.pt \  
  --out-dir data/processed/03_vae_signature_augmented \  
  --include-original \  
  --out-dir data/processed/03_vae_signature_augmented
```



```
--copies-per-target 1 \  
--alpha 1.0 \  
--noise-scale 0.10 \  
--generation-mode residual \  
--residual-scale 0.6
```

Il risultato e' un CSV con 162 righe dati: 81 slice originali e 81 slice sintetiche ottenute trasferendo la firma latente fra i due produttori source.

9.6 Training AdaIN scanner signatures

```
python3 src/training/train_adain_signature.py \  
--train-csv data/processed/01_domain_split/source_train.csv \  
--out-dir experiments/outputs/adain_signature \  
--checkpoint-dir experiments/checkpoints \  
--checkpoint-name adain_signature.pt \  
--epochs 40 \  
--batch-size 8
```

9.7 Generazione immagini con AdaIN

```
python3 src/training/generate_adain_augmented.py \  
--source-csv data/processed/01_domain_split/source_train.csv \  
--adain experiments/checkpoints/adain_signature.pt \  
--out-dir data/processed/05_adain_signature_augmented \  
--include-original \  
--alpha 1.0
```

Il risultato atteso e' un CSV analogo a quello VAE, con 81 slice originali e 81 slice sintetiche prodotte sostituendo le statistiche di stile scanner nelle feature.

9.8 Training CycleGAN

```
python3 src/training/train_cyclegan.py \  
--source-csv data/processed/01_domain_split/source_train.csv \  
--target-csv data/processed/01_domain_split/target_adapt_unlabeled.csv \  
--out-dir experiments/outputs/cyclegan_scanner \  
--checkpoint-dir experiments/checkpoints \  
--epochs 30 \  
--batch-size 4
```

9.9 Traduzione source-to-target

```
python3 -m src.training.translate_source \  
--source-csv data/processed/01_domain_split/source_train.csv \  

```

```
--generator experiments/checkpoints/cyclegan_generator_source_to_target.pt \
--out-dir data/processed/02_translated_source
```

9.10 Training set aumentato VAE+GAN

```
python3 src/datasets/build_augmented_train.py \
--source-csv data/processed/03_vae_signature_augmented/
    source_train_vae_signature_augmented.csv \
--translated-csv data/processed/02_translated_source/translated_source_train.
    csv \
--out-csv data/processed/04_signature_gan_augmented/
    source_train_signature_gan_augmented.csv
```

Il training set aumentato finale contiene 243 righe dati: 81 esempi source originali, 81 esempi sintetici VAE e 81 slice tradotte tramite GAN.

9.11 Training set aumentato AdaIN+GAN

```
python3 src/datasets/build_augmented_train.py \
--source-csv data/processed/05_adain_signature_augmented/
    source_train_adain_signature_augmented.csv \
--translated-csv data/processed/02_translated_source/translated_source_train.
    csv \
--out-csv data/processed/06_adain_gan_augmented/
    source_train_adain_gan_augmented.csv
```

9.12 Training classificatore finale

```
python3 src/training/train_classifier.py \
--train-csv data/processed/04_signature_gan_augmented/
    source_train_signature_gan_augmented.csv \
--val-csv data/processed/01_domain_split/source_val.csv \
--test-csv data/processed/01_domain_split/target_test.csv \
--out-dir experiments/outputs/classifier_signature_gan_augmented \
--checkpoint-dir experiments/checkpoints \
--checkpoint-name classifier_signature_gan_augmented_best_model.pt \
--epochs 20 \
--batch-size 8
```

Per il confronto AdaIN si usa lo stesso protocollo, cambiando solo il CSV di training e la directory di output:

```
python3 src/training/train_classifier.py \
--train-csv data/processed/06_adain_gan_augmented/
    source_train_adain_gan_augmented.csv \
--val-csv data/processed/01_domain_split/source_val.csv \
--test-csv data/processed/01_domain_split/target_test.csv \
```

```
--out-dir experiments/outputs/classifier_adain_gan_augmented \  
--checkpoint-dir experiments/checkpoints \  
--checkpoint-name classifier_adain_gan_augmented_best_model.pt \  
--epochs 20 \  
--batch-size 8
```

9.13 Ambiente di esecuzione

Gli script PyTorch selezionano automaticamente CUDA quando disponibile. Nell'ambiente usato per la run finale, pero', PyTorch non vedeva dispositivi GPU:

```
torch 2.2.2+cu121  
cuda_available False  
device_count 0
```

Di conseguenza il training e' stato eseguito su CPU. Il training VAE e il classificatore finale sono stati completati; per la componente CycleGAN e' stato usato il checkpoint gia' disponibile in `experiments/checkpoints/cyclegan_generator_source_to_target.pt`, poiche' un nuovo training completo su CPU non ha prodotto un checkpoint utilizzabile entro la sessione.

10 Valutazione

La valutazione dei classificatori finali viene salvata nelle rispettive directory sotto `experiments/outputs/`. Lo script `src/evaluation/evaluate_multi_pipeline.py` confronta baseline, CycleGAN, VAE+GAN e AdaIN+GAN e produce:

- `experiments/outputs/evaluation/summary.csv`;
- `experiments/outputs/evaluation/summary.json`;
- `experiments/outputs/evaluation/summary.md`;
- figure in `docs/figures/`.

Le metriche principali sono accuracy e macro-F1. La macro-F1 e' particolarmente utile nel contesto attuale perche' il target test set contiene 12 campioni disease e 23 campioni healthy.

11 Risultati quantitativi

Tabella 3: Risultati sul target test set PHILIPS MEDICAL SYSTEMS. I delta sono calcolati rispetto al baseline source-only.

Modello	Accuracy	Macro-F1	Δ Accuracy	Δ Macro-F1
Source-only	0.6571	0.3966	0.0000	0.0000
CycleGAN-augmented	0.4286	0.4281	-0.2286	0.0316
VAE+GAN-augmented	0.6286	0.3860	-0.0286	-0.0106
AdaIN+GAN-augmented	0.7143	0.6500	0.0571	0.2534

Il modello source-only e' il controllo sperimentale: viene addestrato solo su `source_train.csv`, cioe' su SIEMENS e GE MEDICAL SYSTEMS, e viene valutato su `target_test.csv`, cioe' su PHILIPS MEDICAL SYSTEMS. Il modello CycleGAN-augmented usa un training set che contiene sia source originali sia traduzioni source-to-target. Il modello VAE+GAN-augmented usa un training set da 243 righe dati: 81 slice source originali, 81 slice sintetiche generate tramite firme VAE e 81 slice tradotte tramite GAN. Il modello AdaIN+GAN-augmented usa lo stesso schema, sostituendo le immagini sintetiche VAE con augmentation AdaIN. La valutazione e' stata eseguita solo sul target test set PHILIPS, composto da 35 scansioni non usate per il training.

Dopo l'analisi qualitativa e' stata corretta la generazione VAE signature da decoder diretto a generazione residuale. Le figure qualitative, il CSV aumentato e le metriche multi-modello sono stati aggiornati con la pipeline finale. Il risultato piu' rilevante e' il miglioramento di AdaIN+GAN rispetto al baseline source-only: +0.0571 in accuracy e +0.2534 in macro-F1.

Tabella 4: Metriche per classe del modello source-only. Classe 0: disease, classe 1: healthy.

Classe	Precision	Recall	F1-score	Support
Disease	0.0000	0.0000	0.0000	12
Healthy	0.6571	1.0000	0.7931	23
Macro avg	0.3286	0.5000	0.3966	35

Tabella 5: Dimensione dei dataset intermedi nella pipeline finale.

Artefatto	Righe dati
<code>source_train.csv</code>	81
<code>source_train_vae_signature_augmented.csv</code>	162
<code>translated_source_train.csv</code>	81
<code>source_train_signature_gan_augmented.csv</code>	243
<code>source_train_adain_signature_augmented.csv</code>	162
<code>source_train_adain_gan_augmented.csv</code>	243

12 Scatterplot delle metriche

La Figura 1 e' il grafico generato dalla valutazione multi-modello e salvato direttamente in `docs/figures/08.png`. Il grafico posiziona ogni modello nello spazio accuracy/macro-F1 sul target domain. La Figura 2 e' invece la valutazione source-only vs CycleGAN-augmented mantenuta come artefatto precedente.

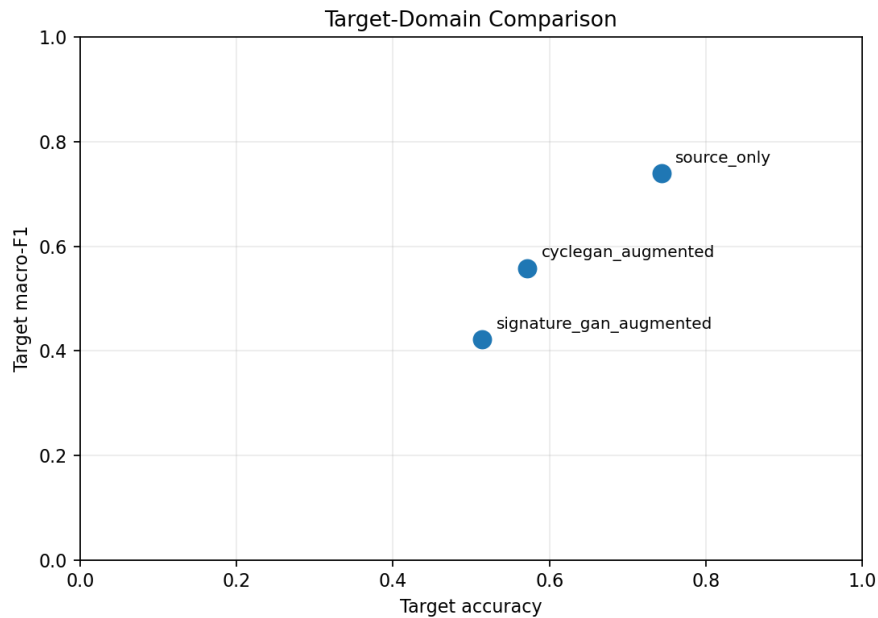


Figura 1: Confronto multi-modello sul target test set PHILIPS: source-only, CycleGAN-augmented, VAE+GAN-augmented e AdaIN+GAN-augmented.

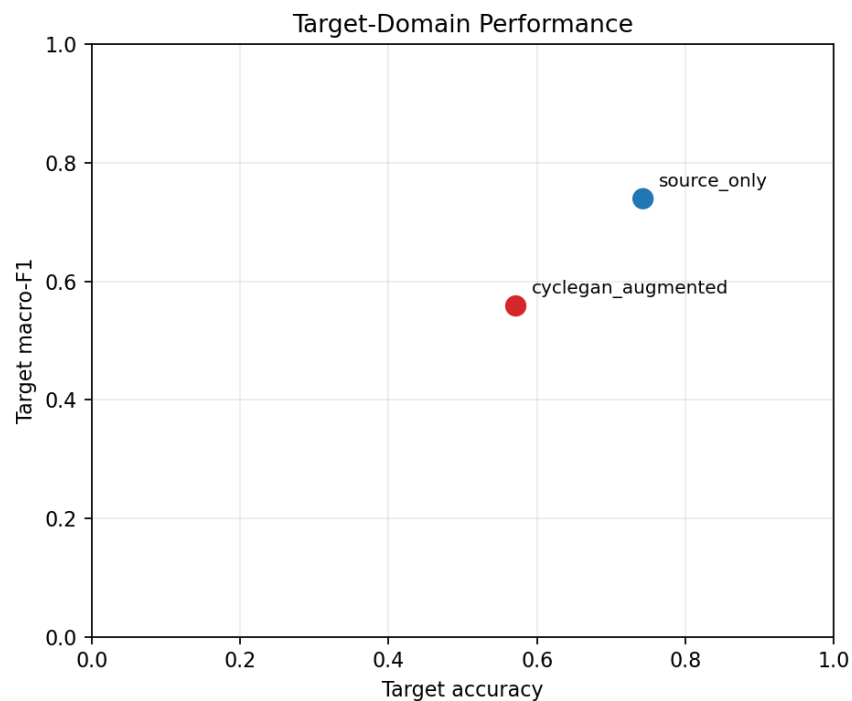


Figura 2: Scatterplot accuracy/macro-F1 dei due modelli valutati sul target test set.

13 Curve temporali del classificatore

La Figura 3 confronta loss, validation accuracy e validation macro-F1 dei training precedenti del classificatore. Nella run finale VAE+GAN, il miglior checkpoint e' stato selezionato sulla validation macro-F1 del source validation set. Il target test set e' rimasto separato fino alla valutazione finale.

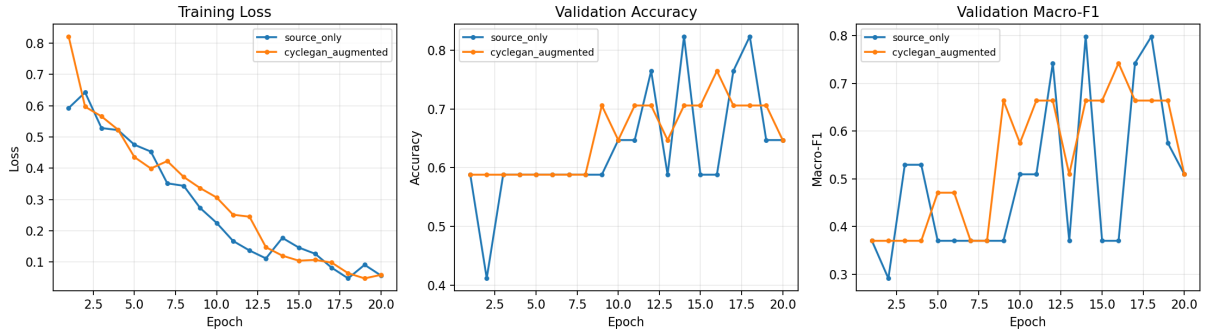


Figura 3: Curve temporali di loss, validation accuracy e validation macro-F1 per esperimenti precedenti source-only e CycleGAN-augmented.

Una possibile interpretazione generale e' che la validazione, essendo ancora nel source domain, non misura direttamente il miglioramento sul target Philips. Il modello puo' apparire buono in validazione source ma non necessariamente generalizzare al target.

14 Curve temporali della CycleGAN

La Figura 4 mostra le loss del generatore e del discriminatore dell'esperimento CycleGAN precedente. Nella run finale e' stato riusato il checkpoint gia' disponibile per generare le traduzioni source-to-target.

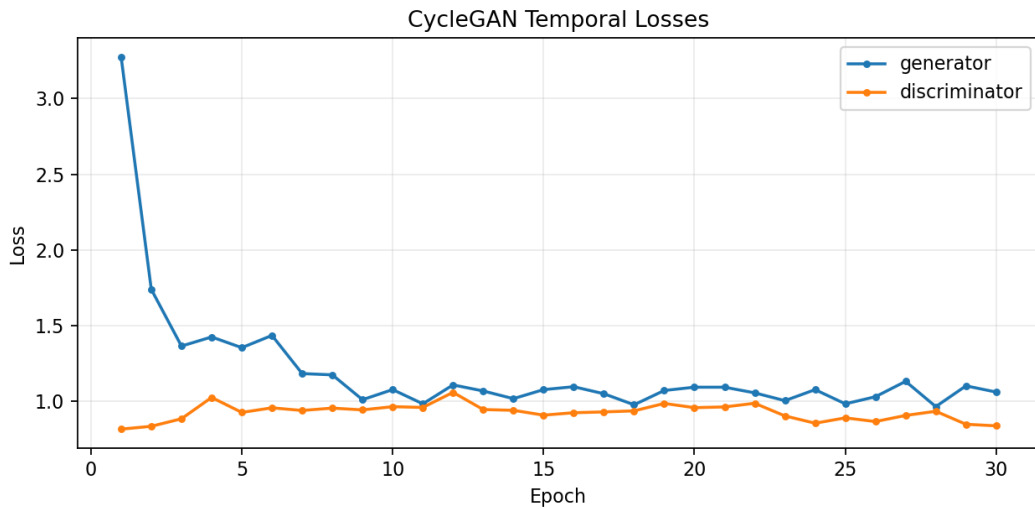


Figura 4: Loss generator/discriminator della CycleGAN durante il training.

La stabilita' numerica della loss non implica automaticamente che la traduzione sia utile per la classificazione. Una CycleGAN puo' produrre immagini visivamente plausibili ma rimuovere o alterare dettagli discriminativi. Nel contesto medico questo e' un rischio importante.

15 Preview qualitative delle immagini modificate

La Figura 5 mostra le preview qualitative aggiornate dopo la correzione della generazione VAE signature. Ogni riga confronta la slice originale, la versione VAE residuale, la traduzione CycleGAN e le rispettive mappe di differenza assoluta. La generazione VAE non usa piu' direttamente il decoder come immagine finale, perche' quel comportamento produceva immagini scure e poco realistiche; ora preserva la slice originale e aggiunge solo il residuale scanner-style stimato dal VAE.

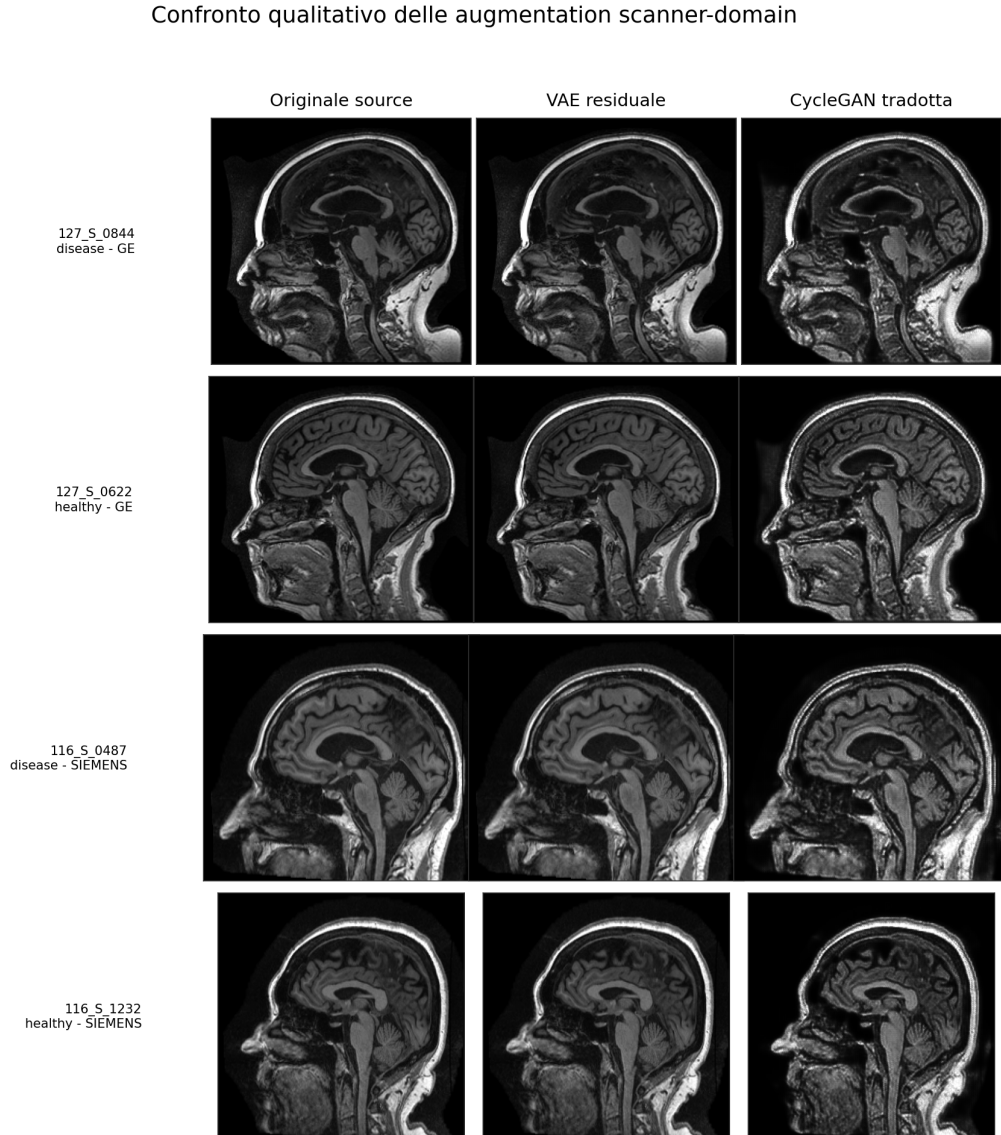


Figura 5: Preview qualitativa aggiornata: originale source, augmentation VAE residuale e traduzione CycleGAN.

La Figura 6 isola le differenze normalizzate per visibilit . Il residuale VAE risulta piu' contenuto rispetto alla traduzione CycleGAN, perche' la nuova procedura e' conservativa e privilegia la preservazione anatomica rispetto alla generazione di una nuova immagine completa.

Mappe di differenza normalizzate: VAE residuale vs CycleGAN

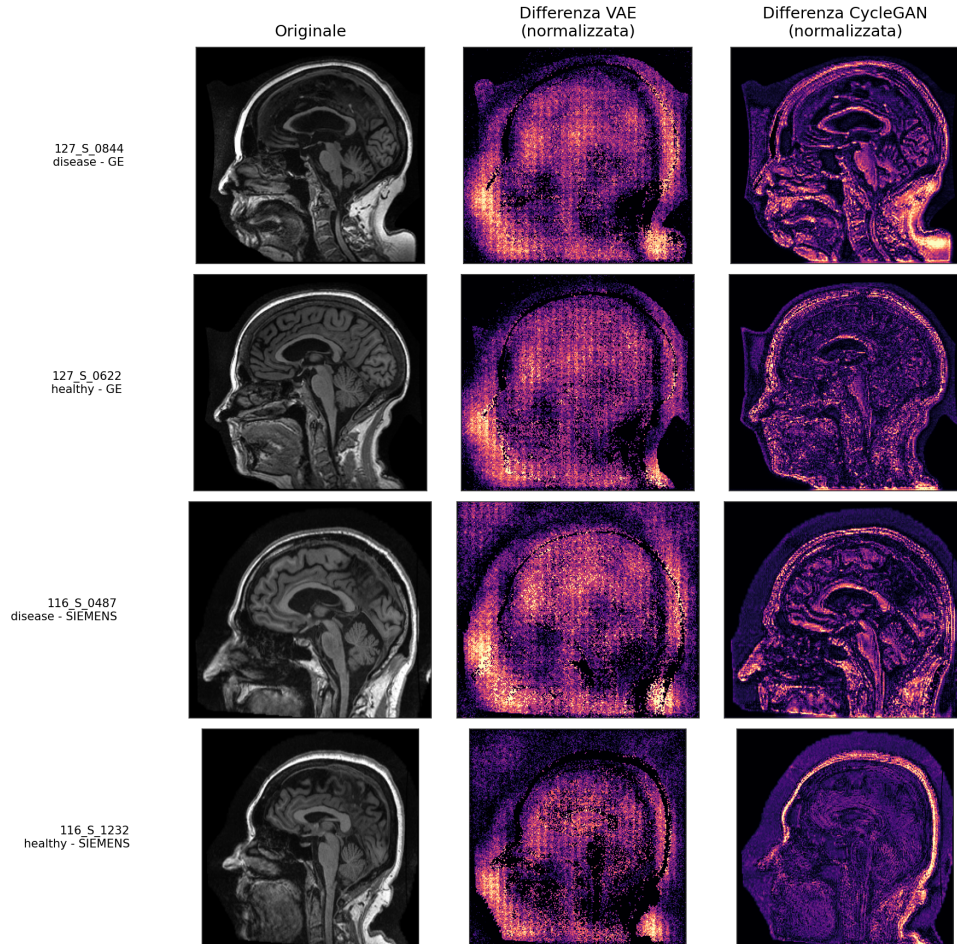


Figura 6: Mappe di differenza normalizzate per VAE residuale e CycleGAN.

Queste preview non costituiscono una validazione clinica, ma permettono di controllare visivamente se le trasformazioni sono localizzate e se introducono artefatti evidenti. La correzione evita il collasso visivo del decoder VAE: l'immagine sintetica conserva l'anatomia dell'originale e applica una perturbazione piu' piccola. Un'analisi piu' rigorosa dovrebbe includere controlli anatomici e metriche quantitative di preservazione strutturale.

16 Figure qualitative aggiuntive

La Figura 7 riporta le preview qualitative della traduzione CycleGAN usate come controllo visuale precedente. La Figura 8 mostra invece la prima versione del confronto fra firma VAE e traduzione GAN, utile per evidenziare il problema della generazione VAE diretta prima della correzione residuale.

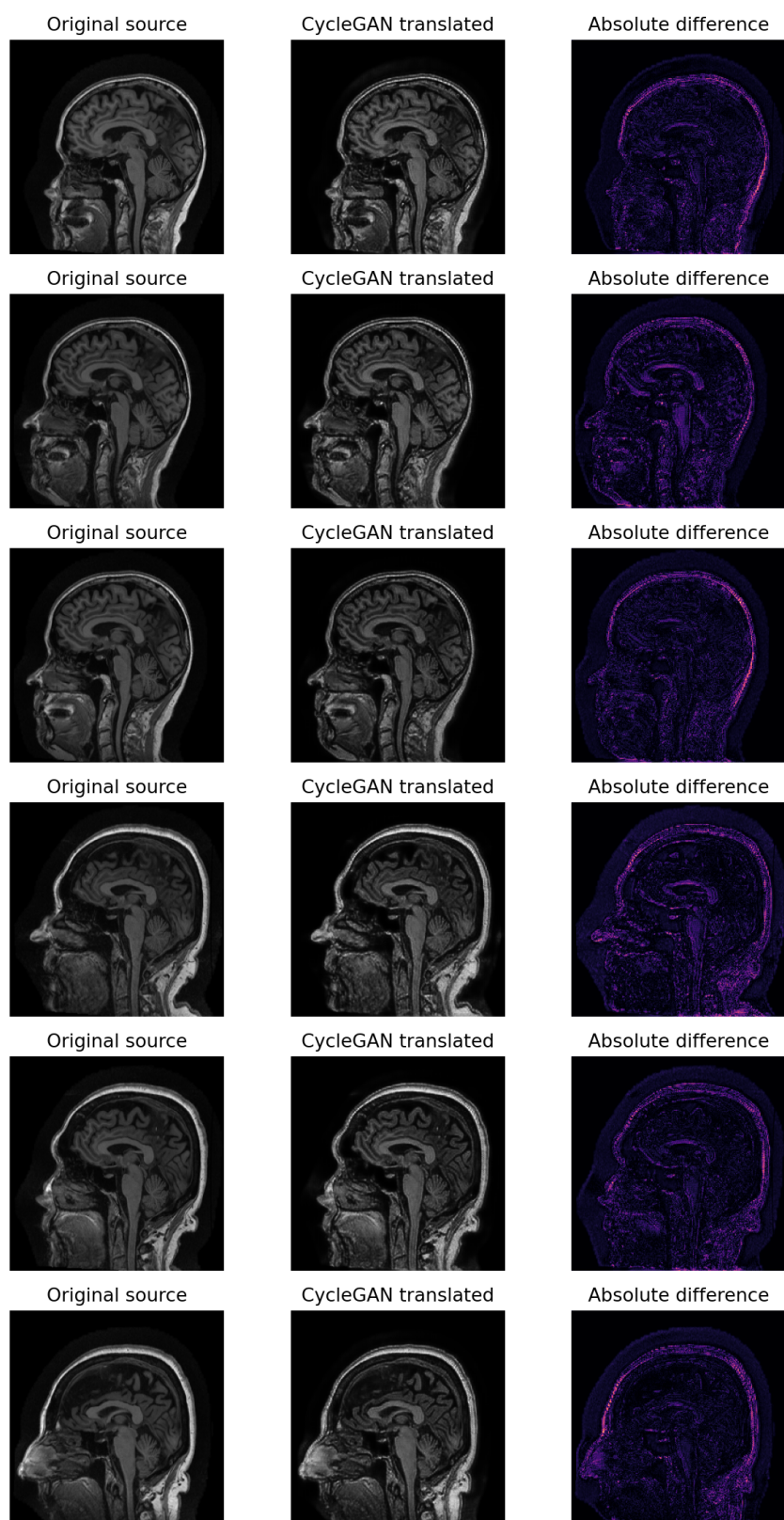


Figura 7: Preview qualitativa precedente della traduzione CycleGAN: slice originale source, slice tradotta e differenza assoluta.

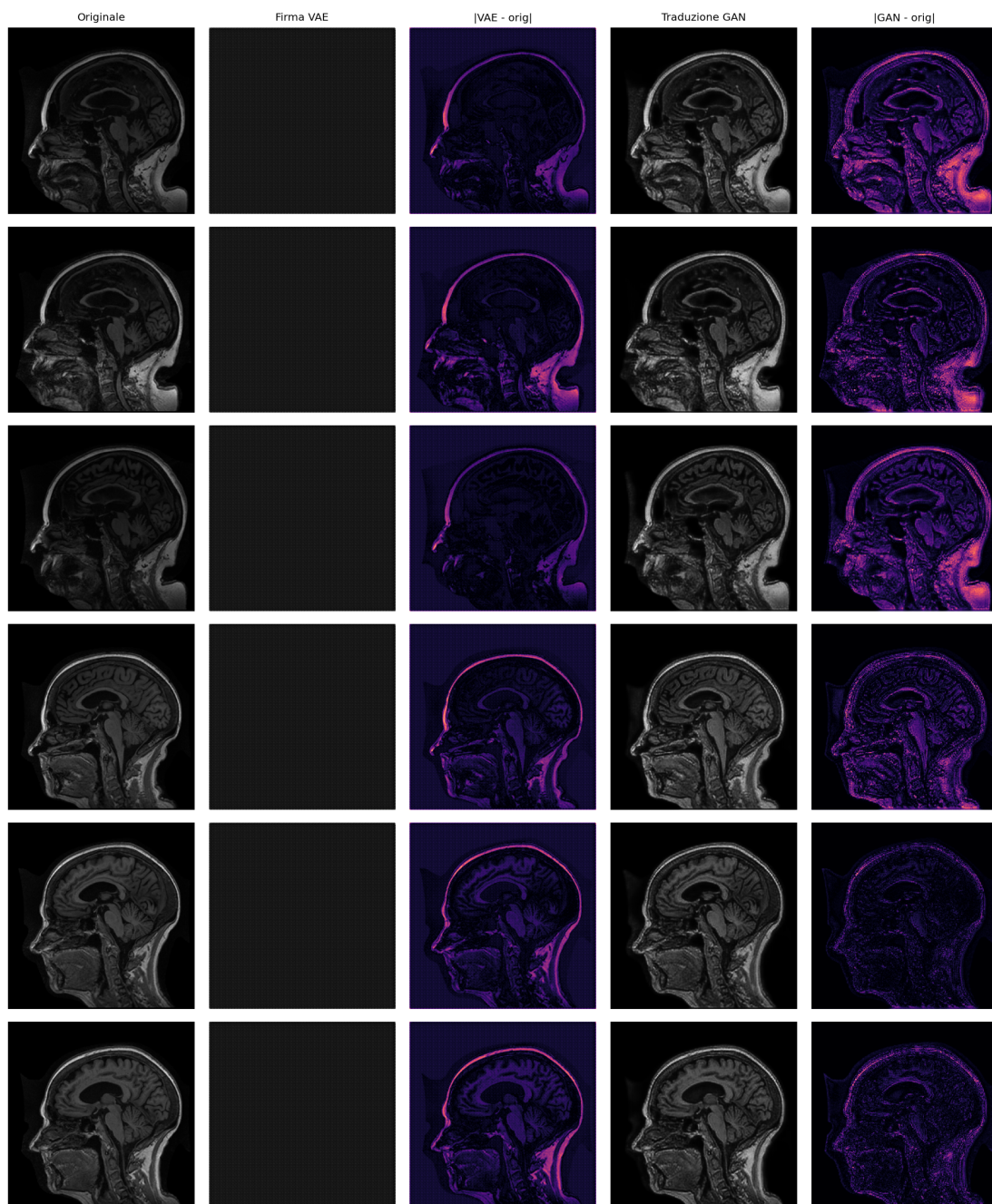


Figura 8: Preview qualitativa precedente del confronto fra firma VAE, traduzione GAN e rispettive differenze rispetto all'originale.

Le Figure 9 e 10 riportano una versione estesa delle preview finali: la prima confronta originale source, augmentation VAE residuale, traduzione CycleGAN e mappe di differenza; la seconda isola le mappe di cambiamento assoluto dopo augmentation generativa.

Esempi qualitativi di augmentation scanner-domain

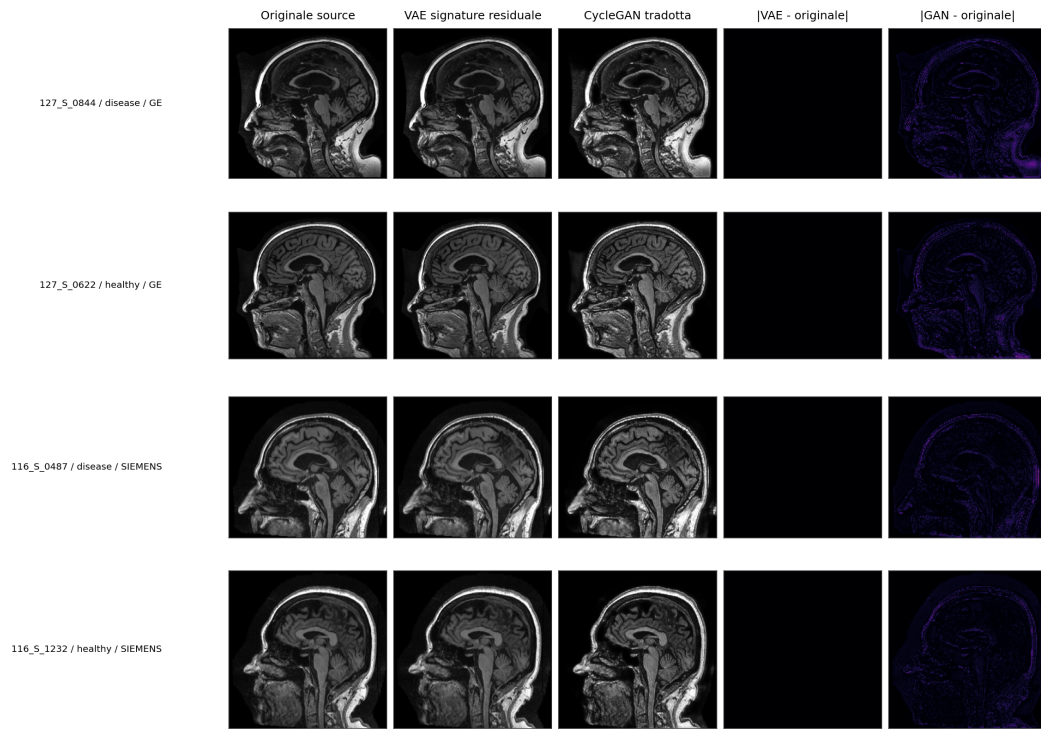


Figura 9: Esempi qualitativi di augmentation scanner-domain con originale source, VAE signature residuale, traduzione CycleGAN e differenze assolute.

Mappe di cambiamento assoluto dopo augmentation generativa

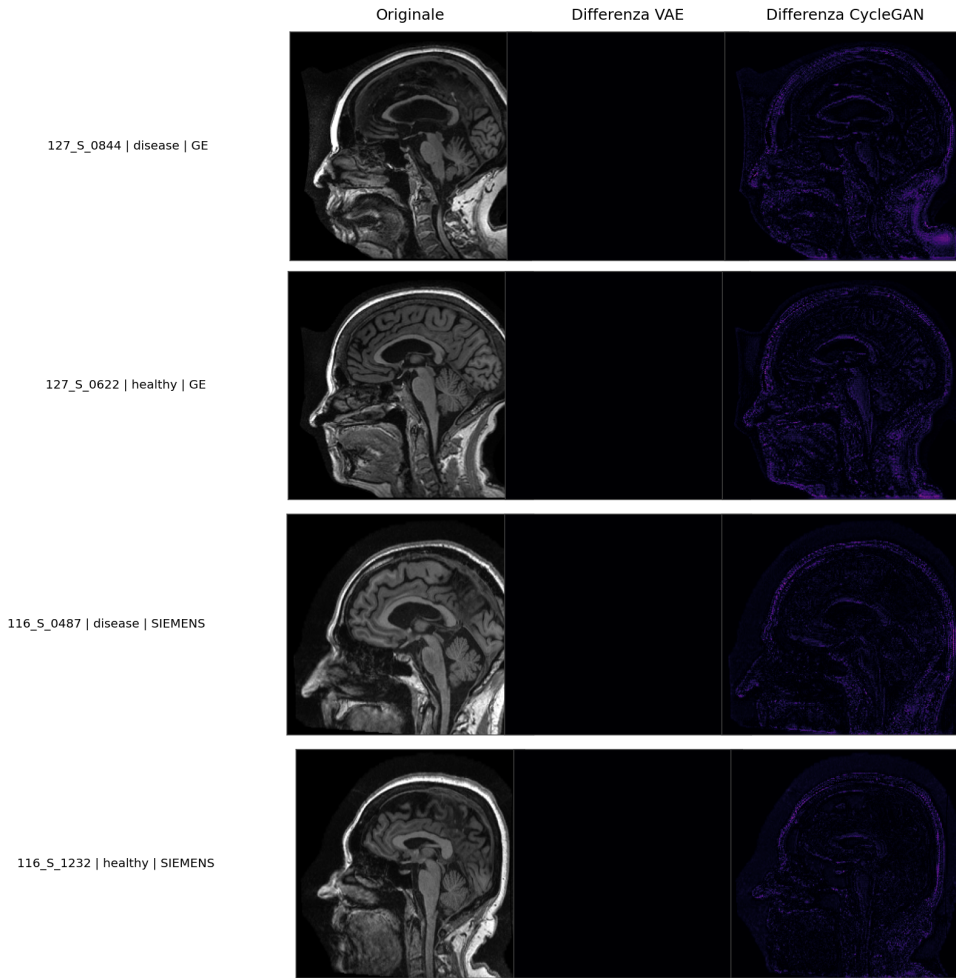


Figura 10: Mappe di cambiamento assoluto dopo augmentation generativa per VAE residuale e CycleGAN.

17 Discussione dei risultati

Il risultato principale della pipeline finale e' che AdaIN+GAN e' il modello migliore sul produttore non visto: accuracy 0.7143 e macro-F1 0.6500. Il baseline source-only ottiene accuracy 0.6571 e macro-F1 0.3966, con un comportamento sbilanciato verso la classe healthy; VAE+GAN resta vicino al baseline in accuracy, ma non in macro-F1; CycleGAN-augmented migliora leggermente la macro-F1 rispetto al baseline, ma perde molta accuracy. Questo indica che la strategia piu' efficace nella run finale e' il trasferimento di statistiche AdaIN combinato con le traduzioni GAN.

Primo, il dataset e' piccolo. Il source train contiene 81 scansioni, e il target adaptation set 36 scansioni. Per addestrare modelli generativi, questi numeri sono molto limitati. Il VAE puo' stimare firme latenti rumorose, mentre la CycleGAN puo' imparare trasformazioni superficiali o instabili invece di una vera mappatura robusta fra domini.

Secondo, la pipeline usa solo la slice centrale di ogni volume. Questa scelta rende il

training piu' leggero, ma riduce drasticamente l'informazione anatomica. La diagnosi CN vs AD puo' dipendere da pattern volumetrici distribuiti, non sempre visibili nella sola slice centrale.

Terzo, le immagini generate vengono usate come augmentation accanto agli originali, ma non c'e' ancora una selezione qualitativa o quantitativa automatica delle traduzioni. La prima versione della generazione VAE usava direttamente il decoder sul vettore latente spostato e produceva immagini troppo scure; il codice e' stato quindi modificato per usare una generazione residuale, cioe' $x_{synthetic} = x + \lambda(Decoder(z') - Decoder(z))$. Questo preserva l'anatomia e rende l'augmentation piu' controllata. Anche la traduzione CycleGAN, pur preservando visivamente la struttura anatomica, puo' modificare dettagli discriminativi o introdurre variazioni non utili alla classificazione.

Quarto, la validazione del classificatore viene fatta su `source_val`, quindi su Siemens e GE. Questo non fornisce una stima diretta delle prestazioni target su Philips. Una possibile estensione sarebbe usare una piccola porzione target etichettata come validation, se disponibile e coerente con il protocollo sperimentale.

Quinto, VAE, AdaIN e CycleGAN non sono vincolati da una loss diagnostica. Essi ottimizzano ricostruzione, statistiche di stile, regolarizzazione latente, plausibilita' visuale e consistenza ciclica, ma non necessariamente preservano caratteristiche legate alla malattia. In ambito medico, questa distinzione e' cruciale: una immagine plausibile dal punto di vista visivo puo' non essere una buona immagine per addestrare un classificatore diagnostico.

18 Checkpoint e artefatti prodotti

I checkpoint sono stati salvati in `experiments/checkpoints`, come richiesto dal README della cartella:

- `classifier_source_only_best_model.pt`;
- `classifier_source_only_best_model_labels.json`;
- `vae_signature.pt`;
- `adain_signature.pt`;
- `cyclegan_generator_source_to_target.pt`;
- `cyclegan_generator_target_to_source.pt`;
- `classifier_cyclegan_augmented_best_model.pt`;
- `classifier_cyclegan_augmented_best_model_labels.json`;
- `classifier_signature_gan_augmented_best_model.pt`;
- `classifier_signature_gan_augmented_best_model_labels.json`;
- `classifier_adain_gan_augmented_best_model.pt`.

Gli output principali della run finale sono:

- experiments/outputs/vae_signature/vae_signature.pt;
- experiments/outputs/vae_signature/scanner_signatures.pt;
- experiments/outputs/vae_signature/manifest.json;
- experiments/outputs/adain_signature/manifest.json;
- data/processed/03_vae_signature_augmented/source_train_vae_signature_augmented.csv;
- data/processed/05_adain_signature_augmented/source_train_adain_signature_augmented.csv;
- data/processed/04_signature_gan_augmented/source_train_signature_gan_augmented.csv;
- data/processed/06_adain_gan_augmented/source_train_adain_gan_augmented.csv;
- experiments/outputs/classifier_source_only/metrics.json;
- experiments/outputs/classifier_cyclegan_augmented/metrics.json;
- experiments/outputs/classifier_signature_gan_augmented/metrics.json;
- experiments/outputs/evaluation_multi/summary.md;
- docs/figures/08.png.

I log sono mantenuti in `experiments/logs`; le figure usate dalla relazione sono in `docs/figures`. Per la pipeline finale e' stata generata anche una nuova figura qualitativa:

- docs/figures/01.png;
- docs/figures/02.png;
- docs/figures/03.png;
- docs/figures/04.png (pipeline CycleGAN precedente);
- docs/figures/05.png (pipeline VAE+GAN precedente);
- docs/figures/06.png (preview qualitativa aggiornata);
- docs/figures/07.png (mappe di differenza aggiornate);
- docs/figures/08.png (confronto quantitativo multi-modello);
- docs/figures/09.png (preview qualitativa estesa);
- docs/figures/10.png (mappe di cambiamento assoluto estese).

19 Riproducibilita'

La configurazione della pipeline e' stata salvata in:

```
experiments/configs/adni_cyclegan_pipeline.json
```

Il file contiene path, iperparametri e directory di output. I principali iperparametri sono:

Tabella 6: Iperparametri principali.

Parametro	Valore
Classifier epochs	20
Classifier batch size	8
Classifier learning rate	0.001
VAE epochs	40
VAE batch size	8
VAE latent dimension	64
VAE beta	0.001
VAE signature alpha	1.0
VAE signature noise scale	0.10
VAE residual scale	0.6
AdaIN epochs	40
AdaIN batch size	8
AdaIN alpha	1.0
CycleGAN epochs	30
CycleGAN batch size	4
CycleGAN learning rate	0.0002
Cycle consistency weight	10.0
Identity weight	2.0

Per riprodurre la pipeline completa, l'ordine corretto e':

1. estrazione archivi in **data/raw**;
2. preparazione ADNI;
3. split con SIEMENS e GE come source e PHILIPS come target;
4. training VAE scanner signatures;
5. generazione immagini con firme sintetiche;
6. training AdaIN scanner signatures;
7. generazione immagini con AdaIN;

8. training o riutilizzo checkpoint CycleGAN;
9. traduzione source;
10. costruzione training set aumentato VAE+GAN;
11. costruzione training set aumentato AdaIN+GAN;
12. training dei classificatori finali VAE+GAN e AdaIN+GAN;
13. evaluation e generazione della figura qualitativa finale.

20 Limiti

Il progetto presenta diversi limiti che devono essere chiariti.

20.1 Dimensione del dataset

Il numero di campioni è ridotto. In particolare, il target test set contiene solo 35 scansioni. Questo rende le metriche sensibili a poche predizioni sbagliate.

20.2 Uso di slice 2D

La pipeline usa la slice centrale del volume, non tutto il volume 3D. Questo semplifica il training ma limita fortemente la rappresentazione anatomica.

20.3 Modelli generativi non supervisionati

Il VAE e la CycleGAN non ricevono informazione diagnostica. Essi possono imparare trasformazioni di intensità, texture o distribuzione latente che non sono necessariamente utili alla classificazione.

20.4 Valutazione qualitativa non clinica

Le preview qualitative mostrano differenze visive, ma non costituiscono una validazione clinica. Per un progetto medico reale servirebbe un controllo più approfondito degli artefatti e della preservazione anatomica.

20.5 Assenza di confronti multipli

La pipeline è stata eseguita con una configurazione principale. Non sono state ancora effettuate ablation su seed, produttori, batch size, numero di epoche, architettura o loss.

21 Possibili estensioni

Le estensioni piu' utili sono:

- usare modelli 3D o 2.5D invece della sola slice centrale;
- aggiungere data augmentation geometrica e di intensita';
- usare una validazione target etichettata se disponibile;
- confrontare VAE+CycleGAN con metodi di feature-level adaptation;
- aggiungere metriche di qualita' delle immagini generate;
- visualizzare embedding prima e dopo adattamento;
- eseguire piu' seed e riportare media e deviazione standard;
- ruotare i produttori e usare a turno Siemens, GE e Philips come dominio di test;
- introdurre early stopping basato su una metrica piu' coerente con il target domain.

22 Conclusione

Il progetto ha prodotto una pipeline completa e riproducibile per studiare domain adaptation su MRI ADNI. Sono stati implementati preprocessing, split patient-level per dominio scanner, VAE per firme latenti di macchinari, AdaIN per statistiche di stile scanner, generazione di immagini sintetiche, traduzione CycleGAN, training aumentato VAE+GAN e AdaIN+GAN, evaluation quantitativa e nuove figure qualitative della pipeline finale.

Il risultato finale e' che il modello source-only addestrato su SIEMENS e GE MEDICAL SYSTEMS raggiunge accuracy 0.6571 e macro-F1 0.3966 sul target test set PHILIPS MEDICAL SYSTEMS. Il modello CycleGAN-augmented ottiene 0.4286 e 0.4281, il modello VAE+GAN-augmented ottiene 0.6286 e 0.3860, mentre AdaIN+GAN-augmented raggiunge 0.7143 e 0.6500. Questo fornisce un risultato sperimentale chiaro: nella configurazione attuale, il trasferimento AdaIN combinato con GAN e' la variante piu' efficace, anche se il dataset piccolo, l'uso di sole slice centrali e il training generativo non supervisionato restano limiti importanti.

Il lavoro svolto costituisce quindi una base solida per esperimenti successivi. La struttura del codice e' stata organizzata secondo le responsabilita' del repository, gli artefatti sono stati salvati nelle cartelle corrette e le figure nuove sono state copiate in docs/figures/, in particolare docs/figures/06.png, docs/figures/07.png, docs/figures/08.png, docs/figures/09.png e docs/figures/10.png.

A Comandi principali e spiegazione operativa

Questa appendice riporta la pipeline nello stesso ordine in cui deve essere eseguita. Ogni comando produce artefatti usati dalla fase successiva; per questo i path di input e output sono parte integrante del protocollo sperimentale.

A.1 Accesso al cluster e ambiente GPU

Questi comandi richiedono una sessione interattiva Slurm con una GPU e poi aprono il container Apptainer con supporto NVIDIA. Il flag `-nv` e' essenziale: senza di esso PyTorch puo' non vedere CUDA anche se il job Slurm ha una GPU assegnata.

```
srunk --account=dl-course-q2 --partition=dl-course-q2 --qos=gpu-medium --gres=gpu
:1 --pty bash
cd ~/DomainAdaptation-Track8-EventHorizon
apptainer shell --nv /shared/sifs/latest.sif
nvidia-smi
python -c "import torch; print(torch.__version__, torch.cuda.is_available(),
    torch.cuda.device_count())"
```

A.2 Esecuzione automatica completa

Lo script `src/training/run_cluster_vae_adain_pipeline.sh` incapsula l'intero flusso. Prima controlla CUDA e le dipendenze Python, poi crea split, baseline, CycleGAN, VAE, AdaIN, classifier finali ed evaluation. Ogni step viene eseguito tramite la funzione Bash `run_step`, che salva anche il log in `experiments/logs/NOME_STEP.log`.

```
bash src/training/run_cluster_vae_adain_pipeline.sh
```

Variabili utili:

```
RUN_PREPROCESS=auto bash src/training/run_cluster_vae_adain_pipeline.sh
RUN_CYCLEGAN=0 bash src/training/run_cluster_vae_adain_pipeline.sh
CLASSIFIER_EPOCHS=1 GEN_EPOCHS=1 CYCLEGAN_EPOCHS=1 bash src/training/
    run_cluster_vae_adain_pipeline.sh
```

`RUN_PREPROCESS=auto` salta il preprocessing se `index_prepared.csv` esiste gia'.
`RUN_CYCLEGAN=0` riusa il checkpoint CycleGAN esistente. L'ultima riga e' una smoke test rapida: serve solo a verificare che il flusso parta, non a produrre risultati scientifici.

A.3 Preparazione ADNI

Questo script legge il CSV ADNI, trova i file NIfTI e XML, filtra le scansioni T1 a 3T, estrae i metadati scanner e salva i volumi normalizzati in formato NumPy. L'output centrale e' `index_prepared.csv`, che diventa la tabella master per tutti gli step successivi.

```
python src/datasets/prepare_adni.py --csv data/ADNI1_Annual_2_Yr_3T_5_26_2026.csv
--nifti-root data/raw/nifti --metadata-root data/raw/metadata --out-dir data
/processed/00_prepared --field 3T --label-mode cn_vs_ad
```

Input principali: CSV ADNI, cartella NIfTI e cartella XML. Output principali: data/processed/00_prepared/index_prepared.csv e volumi .npy normalizzati.

A.4 Split patient-level per dominio scanner

Questo comando divide il dataset per produttore. SIEMENS e GE MEDICAL SYSTEMS sono usati come source domain, PHILIPS MEDICAL SYSTEMS come target domain. Lo split e' patient-level: lo stesso soggetto non puo' finire contemporaneamente in train e test.

```
python src/datasets/make_domain_splits.py --index-csv data/processed/00_prepared/
index_prepared.csv --source-manufacturers "SIEMENS,GE_MEDICAL_SYSTEMS" --
target-manufacturer "PHILIPS_MEDICAL_SYSTEMS" --out-dir data/processed/01
_domain_split
```

Output: source_train.csv, source_val.csv, target_adapt_unlabeled.csv e target_test.csv. Il target adaptation set puo' essere usato senza label per il training generativo; il target test set resta separato fino alla valutazione finale.

A.5 Baseline source-only

Il baseline addestra il classificatore solo sui dati source originali. Serve come controllo: se una tecnica di augmentation e' utile, dovrebbe superare questo modello sul target PHILIPS.

```
python src/training/train_classifier.py --train-csv data/processed/01
_domain_split/source_train.csv --val-csv data/processed/01_domain_split/
source_val.csv --test-csv data/processed/01_domain_split/target_test.csv --
out-dir experiments/outputs/classifier_source_only --checkpoint-dir
experiments/checkpoints --checkpoint-name classifier_source_only_best_model.
pt --epochs 20 --batch-size 8
```

Output: checkpoint in experiments/checkpoints/classifier_source_only_best_model.pt e metriche in experiments/outputs/classifier_source_only/metrics.json.

A.6 CycleGAN e traduzione source-to-target

La CycleGAN impara una traduzione non appaiata fra source e target usando source train e target adaptation unlabeled. Vengono addestrati due generatori, ma per costruire il training set aumentato si usa soprattutto cyclegan_generator_source_to_target.pt.

```
python src/training/train_cyclegan.py --source-csv data/processed/01
_domain_split/source_train.csv --target-csv data/processed/01_domain_split/
target_adapt_unlabeled.csv --out-dir experiments/outputs/cyclegan_scanner --
checkpoint-dir experiments/checkpoints --epochs 30 --batch-size 4
```

```
python -m src.training.translate_source --source-csv data/processed/01
_domain_split/source_train.csv --generator experiments/checkpoints/
cyclegan_generator_source_to_target.pt --out-dir data/processed/02
_translated_source
```

Il primo comando produce i checkpoint dei generatori e i log della loss. Il secondo applica il generatore source-to-target a ogni esempio source e salva `data/processed/02_translated_source/translated_source_train.csv`.

A.7 Training del classificatore CycleGAN-augmented

Prima si fonde il source originale con le immagini tradotte, poi si addestra un classificatore con lo stesso protocollo del baseline.

```
python src/datasets/build_augmented_train.py --source-csv data/processed/01
_domain_split/source_train.csv --translated-csv data/processed/02
_translated_source/translated_source_train.csv --out-csv data/processed/02
_translated_source/source_train_augmented.csv

python src/training/train_classifier.py --train-csv data/processed/02
_translated_source/source_train_augmented.csv --val-csv data/processed/01
_domain_split/source_val.csv --test-csv data/processed/01_domain_split/
target_test.csv --out-dir experiments/outputs/classifier_cyclegan_augmented
--checkpoint-dir experiments/checkpoints --checkpoint-name
classifier_cyclegan_augmented_best_model.pt --epochs 20 --batch-size 8
```

Output: `experiments/outputs/classifier_cyclegan_augmented/metrics.json`.

A.8 VAE scanner signatures

Il VAE apprende una rappresentazione latente delle slice source. Dopo il training, lo script calcola una firma media per produttore nello spazio latente. La generazione usa la direzione tra firma source e firma target per creare varianti sintetiche.

```
python src/training/train_vae_signature.py --train-csv data/processed/01
_domain_split/source_train.csv --out-dir experiments/outputs/vae_signature --
checkpoint-dir experiments/checkpoints --checkpoint-name vae_signature.pt --
epochs 40 --batch-size 8

python src/training/generate_signature_augmented.py --source-csv data/processed
/01_domain_split/source_train.csv --vae experiments/checkpoints/vae_signature.
pt --out-dir data/processed/03_vae_signature_augmented --include-original --
copies-per-target 1 --alpha 1.0 --noise-scale 0.10 --generation-mode residual
--residual-scale 0.6
```

Il parametro `-include-original` mantiene anche le immagini reali nel CSV aumentato. `-copies-per-target 1` genera una variante sintetica per ogni esempio. `-alpha` controlla l'intensità dello spostamento latente, mentre `-noise-scale` aggiunge rumore controllato

nel latent space. La modalita' `-generation-mode residual` evita di salvare direttamente il decoder VAE come immagine finale: salva invece la slice originale piu' il residuale di stile $\lambda(Decoder(z') - Decoder(z))$, controllato da `-residual-scale`.

A.9 Training set VAE+GAN e classificatore finale

Qui vengono fusi tre blocchi: source originali, immagini generate dal VAE e traduzioni CycleGAN. Il classificatore finale e' poi addestrato sul CSV combinato.

```
python src/datasets/build_augmented_train.py --source-csv data/processed/03
_vae_signature_augmented/source_train_vae_signature_augmented.csv --
translated-csv data/processed/02_translated_source/translated_source_train.
csv --out-csv data/processed/04_signature_gan_augmented/
source_train_signature_gan_augmented.csv

python src/training/train_classifier.py --train-csv data/processed/04
_signature_gan_augmented/source_train_signature_gan_augmented.csv --val-csv
data/processed/01_domain_split/source_val.csv --test-csv data/processed/01
_domain_split/target_test.csv --out-dir experiments/outputs/
classifier_signature_gan_augmented --checkpoint-dir experiments/checkpoints
--checkpoint-name classifier_signature_gan_augmented_best_model.pt --epochs
20 --batch-size 8
```

Output: `experiments/outputs/classifier_signature_gan_augmented/metrics.json`. Questo e' il modello indicato nelle tabelle come VAE+GAN-augmented.

A.10 Ramo AdaIN predisposto

Il ramo AdaIN segue la stessa logica del VAE, ma trasferisce statistiche di feature invece di spostare vettori latenti globali. Il classificatore AdaIN+GAN e' confrontabile solo quando il relativo `metrics.json` viene prodotto.

```
python src/training/train_adain_signature.py --train-csv data/processed/01
_domain_split/source_train.csv --out-dir experiments/outputs/adain_signature
--checkpoint-dir experiments/checkpoints --checkpoint-name adain_signature.pt
--epochs 40 --batch-size 8

python src/training/generate_adain_augmented.py --source-csv data/processed/01
_domain_split/source_train.csv --adain experiments/checkpoints/
adain_signature.pt --out-dir data/processed/05_adain_signature_augmented --
include-original --alpha 1.0

python src/datasets/build_augmented_train.py --source-csv data/processed/05
_adain_signature_augmented/source_train_adain_signature_augmented.csv --
translated-csv data/processed/02_translated_source/translated_source_train.
csv --out-csv data/processed/06_adain_gan_augmented/
source_train_adain_gan_augmented.csv

python src/training/train_classifier.py --train-csv data/processed/06
_adain_gan_augmented/source_train_adain_gan_augmented.csv --val-csv data/
```

```
processed/01_domain_split/source_val.csv --test-csv data/processed/01_domain_split/target_test.csv --out-dir experiments/outputs/classifier_adain_gan_augmented --checkpoint-dir experiments/checkpoints --checkpoint-name classifier_adain_gan_augmented_best_model.pt --epochs 20 --batch-size 8
```

A.11 Valutazione e figure in docs/figures/

La valutazione a due modelli genera `experiments/outputs/evaluation/summary.md`. La valutazione multi-modello genera `experiments/outputs/evaluation_multi/summary.md` e il grafico `docs/figures/08.png`. Tutte le figure usate dal report devono rimanere nella cartella `docs/figures/`, così la relazione può includerle con un path unico.

```
python src/evaluation/evaluate_pipeline.py --baseline experiments/outputs/classifier_source_only/metrics.json --augmented experiments/outputs/classifier_cyclegan_augmented/metrics.json --out-dir experiments/outputs/evaluation --figures-dir docs/figures

python src/evaluation/evaluate_multi_pipeline.py --model source_only=experiments/outputs/classifier_source_only/metrics.json --model cyclegan_augmented=experiments/outputs/classifier_cyclegan_augmented/metrics.json --model vae_gan_augmented=experiments/outputs/classifier_signature_gan_augmented/metrics.json --model adain_gan_augmented=experiments/outputs/classifier_adain_gan_augmented/metrics.json --out-dir experiments/outputs/evaluation_multi --figures-dir docs/figures
```

Per leggere i risultati:

```
cat experiments/outputs/evaluation/summary.md
cat experiments/outputs/evaluation_multi/summary.md
ls -lh docs/figures
```

Riferimenti bibliografici

- [1] Zhu, J.-Y., Park, T., Isola, P., Efros, A. A. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. ICCV, 2017.
- [2] Alzheimer's Disease Neuroimaging Initiative. *ADNI: Alzheimer's Disease Neuroimaging Initiative*.
- [3] Ganin, Y., et al. *Domain-Adversarial Training of Neural Networks*. Journal of Machine Learning Research, 2016.